

GoldenGuide

A grounded academic assistant for Wilfrid Laurier University

Software Design Document

Author	Basleal Abshiro (b4sleal)
Version	1.5
Date	November 2, 2026
Status	Describes the implemented system
Repository	https://github.com/b4sleal/GoldenGuide
Live system	https://askgoldenguide.ca

Revision History

Version	Date	Author	Changes
1.0	2026-10-30	Basleal Abshiro	Initial release describing the implemented system: collection, storage, indexing, embedding service, question answering, security, configuration, observability, deployment and verification.
1.1	2026-10-30	Basleal Abshiro	Added the staging environment and per-environment configuration files; corrected the description of retrieval profile fallback (semantic search only; no answer-model fallback configured).
1.2	2026-10-30	Basleal Abshiro	Staging and production run their own Keycloak with a hosted realm that seeds the first administrator; each environment has its own Compose project name and data volumes; the pre-release database migrations were folded into a single baseline; the API documentation's inline script is allowed by hash in the content security policy.
1.3	2026-10-31	Basleal Abshiro	Multiple machines over Tailscale: each machine's `tailscale` container is its own private device and forwards that machine's services by name, so a stack can be split by role (data node, application node) with one env file per machine; the `services` profile groups the shared data stores and Keycloak; env files choose their Compose files and profiles, so commands no longer repeat them; the backend waits for a healthy Keycloak; the IDE workspace is part of the development stack.
1.4	2026-11-01	Basleal Abshiro	React web client replaces the static page (new Web Client chapter and decision D13); stored conversations with follow-up memory, answer snapshots and feedback (tables `conversations`, `chat_messages`, `message_feedback`); conversation and feedback endpoints; `status` stream event; frontend tests; roadmap updated.
1.5	2026-11-02	Basleal Abshiro	ColBERT reorders the dense and sparse candidates instead of searching the whole collection, so its token vectors have no HNSW index (indexing took hours); search filters apply to every candidate search; completed answers are stored before the stream ends and a `saved` stream event gives the client their message ID, so every answer can be rated (risk R15 closed); the answer prompt repeats the citation rule after the context; the `sources` event is sent before the answer text; Academic Calendar pages are typed by URL first (department pages and faculty sub-pages were stored as faculties, and graduate Public Safety courses as faculty pages), and departments are stored under the lexicon's canonical name; courses and WLU pages carry an `academic_level` (migration `0006`, filled from calendar URLs) also in the vector payload; calendar re-imports update record metadata; indexing runs accept `payload_only`, which rewrites the payload of existing vectors without embedding.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Purpose

This document describes the software design of GoldenGuide: what the system must do, how it is structured to do it, and why each significant design choice was made. It is the reference for anyone who maintains, extends, reviews or operates GoldenGuide. It follows the structure of IEEE 1016 (Software Design Descriptions): context, composition, logical, information, interface, interaction and deployment viewpoints, with design rationale recorded alongside them.

1.2 Problem statement

Laurier students find academic information in several systems that do not talk to each other:

- the **WLU websites** (policies, student services, deadlines, financial information);
- the **Academic Calendar** (courses, programs, requirements, regulations) for each academic year;
- **LORIS**, the registration system (term offerings: sections, CRNs, meeting times, seats, waitlists).

The existing Service Laurier assistant only answers pre-written questions. A question such as *"Can I take CP264 this winter, and when does it meet?"* requires the student to consult at least two systems and understand how a calendar course relates to a term's sections.

1.3 Goal

A single conversational assistant that answers from all three official sources, cites every claim with a working link, and states clearly when it cannot answer, built to production standards so it can be operated, maintained and extended over the long term.

1.4 Scope

Table 1.1: Scope

In scope	Out of scope
Answering questions from official WLU sources, with citations	Registering students or changing any WLU record
Collecting the Academic Calendar, WLU websites and LORIS on a schedule	Degree-planning advice beyond what sources state
Storing raw sources, structured records and vector indexes	Third-party ratings or community commentary as evidence
Administration: collection, indexing, job status, runtime settings, uploads	A replacement for LORIS or a full student portal
Sign-in with explicit roles, ready for WLU's identity provider	

1.5 Intended audience

Software engineers working on GoldenGuide; reviewers assessing its architecture; and technical readers interested in the design of grounded retrieval-augmented generation (RAG) systems.

1.6 Document overview

Chapter **??** lists requirements and constraints. Chapter **??** presents the architecture and Chapter **??** the design decisions with their alternatives. Chapters **??**–**??** give the detailed design of each subsystem, from crawling and storage to question answering and the web client. Chapters **??**–**??** cover data, interfaces and interactions. Chapters **??**–**??** cover quality attributes, deployment, verification, risks and the roadmap. Appendices contain a glossary, the repository layout and the configuration catalogue.

2 Requirements and Constraints

2.1 Functional requirements

Table 2.1: Functional requirements

ID	Requirement
F1	Answer natural-language questions about courses, term offerings, prerequisites, registration, programs, policies, deadlines and services.
F2	Answer factual lookups (prerequisites, sections, meeting times, rooms, instructors, seats) exactly as recorded in the source.
F3	Cite the sources behind every answer with working links to the original pages.
F4	Distinguish *no records found*, *operation unsupported*, *a source failed* and *results were limited* in the answer.
F5	Resolve relative academic terms (“next term”, “this fall”) to concrete terms from the current date, in code.
F6	Stream answers progressively, ending with an explicit completion or failure signal.
F7	Crawl the Academic Calendar and WLU websites, and download LORIS term offerings, storing raw sources.
F8	Parse stored sources into structured records: courses, course terms (sections), programs and pages.
F9	Index records for semantic search (chunk, enrich, embed) and re-index only what changed.
F10	Run every collection and indexing step as an independently runnable, tracked, retryable job.
F11	Refresh all sources nightly without manual intervention.
F12	Let administrators run jobs, view their status, change runtime settings and upload documents.
F13	Sign users in through an institutional OpenID Connect provider with explicitly mapped roles.
F14	Let automation call administrative operations with narrowly scoped, revocable credentials.
F15	Keep conversations so students can return to them, ask follow-up questions that refer to earlier turns, and rate answers.

2.2 Non-functional requirements

Table 2.2: Non-functional requirements

ID	Quality	Requirement
N1	Trustworthiness	Never fabricate courses, terms, prerequisites, policies or links; the answer model sees only retrieved evidence.
N2	Provenance	Keep the source of every record; Academic Calendar and LORIS data never silently overwrite each other.
N3	Reliability	One failing source degrades part of an answer, not all of it; failed ingestion work is retried without redoing successful work.
N4	Security	Administrative operations require authorization; failures of sign-in or session state deny access; secrets never appear in code, YAML, images or logs.
N5	Scalability	The API runs as multiple replicas without process-local shared state; services can run on separate machines.
N6	Observability	Every request is traceable end to end through logs and traces by a single identifier.
N7	Maintainability	Clear layer boundaries, typed configuration, provider replacement without changing use cases, behavioural tests.
N8	Operability	One-command local setup with only Docker; bounded CPU and memory per service; reproducible builds.
N9	Cost control	Use deterministic logic instead of an LLM where possible; bound prompt sizes; rate-limit public chat.
N10	Politeness	Crawling respects `robots.txt`, uses conditional requests and bounded concurrency.

2.3 Constraints

- **Public sources only.** GoldenGuide reads public WLU pages and the public LORIS class search like a browser would. Their structure can change without notice.
- **No WLU system integration yet.** There is no data agreement with WLU; sign-in through WLU's Microsoft Entra ID tenant requires WLU to register the application.
- **External, paid, fallible LLMs.** Their output is untrusted input and their availability is not guaranteed.
- **Limited hardware.** Development and current production run on one PC with a consumer NVIDIA GPU; the system must also work CPU-only.
- **Parallel development.** Layer boundaries, ports and separately deployable services let several developers work on different parts (collection, retrieval, API, infrastructure) without stepping on each other; explicit structure and documentation are preferred over cleverness.

2.4 Assumptions

- The Academic Calendar is published per academic year and is authoritative for course descriptions.
- LORIS is authoritative for term-specific sections and may differ from the calendar.
- Students accept answers that say part of a question could not be checked, provided they are told why.

3 Architecture

3.1 Architectural style

GoldenGuide is a **modular monolith**: one FastAPI backend containing the API and the background jobs, plus a separate **embedding service**, with independently operated data stores. Each runs in its own container with its own resource limits and Compose profile, so any of them can be moved to another machine. Inside the backend, a **hexagonal (ports and adapters)** architecture with domain-driven design keeps business logic independent of frameworks and vendors.

3.2 System context

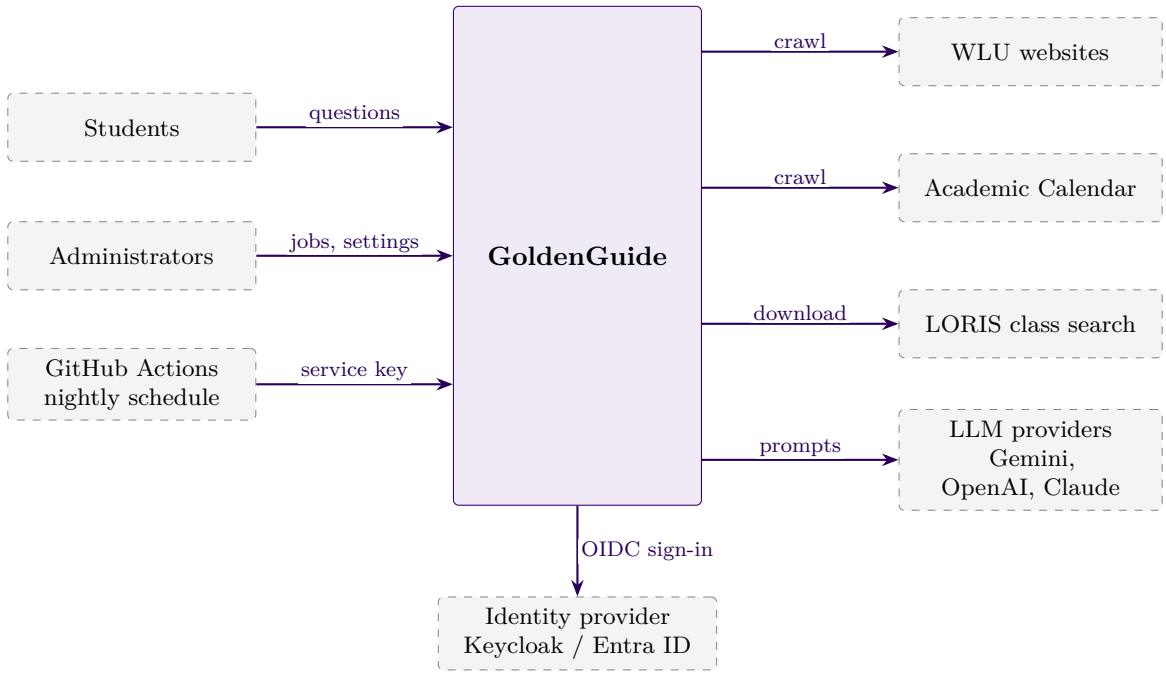


Figure 3.1: System context

3.3 Container view

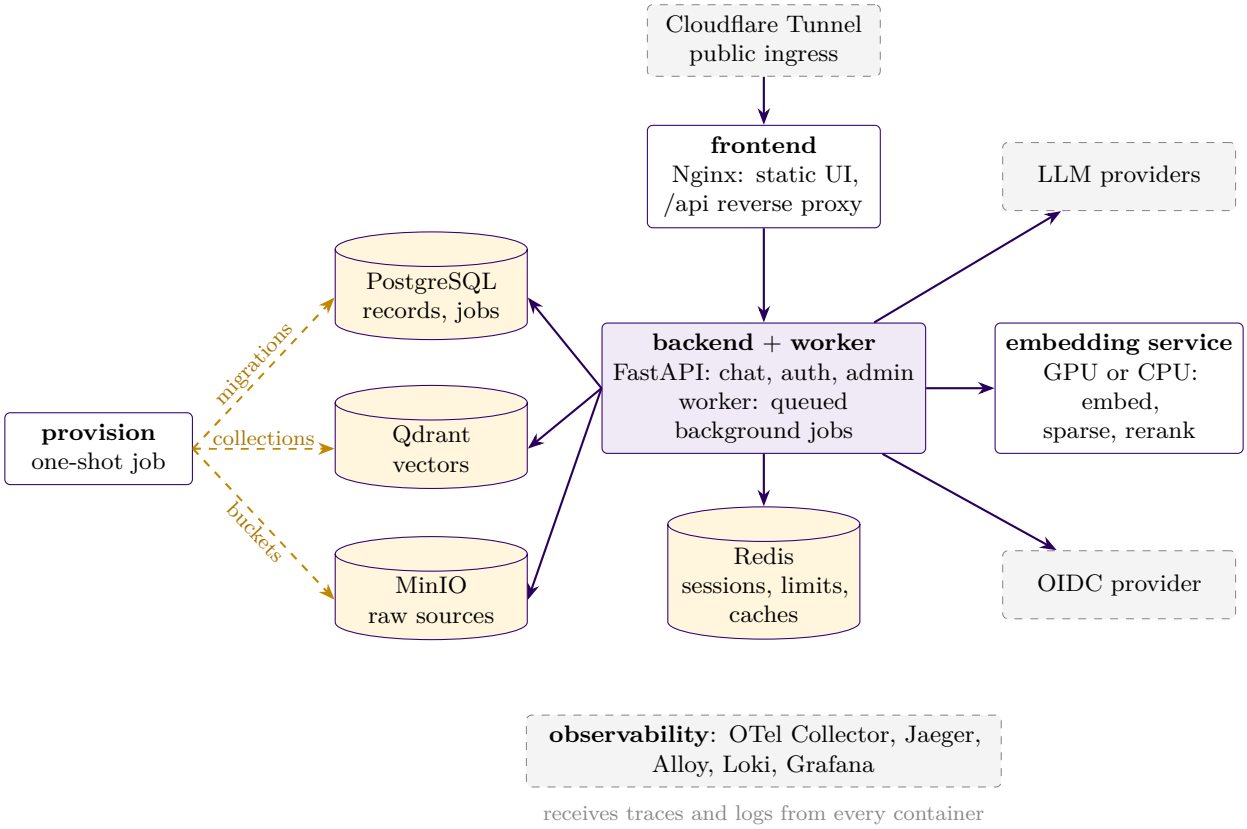


Figure 3.2: Container view

Table 3.1: Containers

Container	Built from	Responsibility
frontend	Nginx image	Serves the static UI and proxies `/api`, `/docs` and health routes to the backend, unbuffered for streaming. The browser only talks to this origin.
backend	Python runtime image (backend group)	FastAPI API (chat, sign-in, admin). Records and queues background jobs (runs them in-process only in development).
worker	Same image as backend	Runs queued background jobs (crawling, parsing, indexing) with its own CPU and memory limits; parses in a pool of child processes.
provision	Same image as backend	One-shot job before the API starts: Alembic migrations, MinIO buckets, Qdrant collections with version checks. Idempotent.

Container	Built from	Responsibility
local-embedding-gpu / -cpu	Python runtime image (embedding group)	Separate service loading dense, sparse, multi-vector and reranking models; API-key protected.
postgres + db-bootstrap	postgres:16	Authoritative structured data; the bootstrap job creates the databases.
qdrant	Locked version	Vector collections.
minio	Locked version	S3-compatible object storage for raw sources and uploads.
redis	redis:7-alpine	Critical session, OAuth and rate-limit state (append-only persistence) and optional caches.
keycloak	Keycloak 26	OpenID Connect provider: a development realm locally, and a hosted realm (one Keycloak per environment) for staging and production.
observability	Upstream images	OpenTelemetry Collector, Jaeger, Alloy, Loki, Grafana.
tailscale, tunnel	Upstream images	Private network between machines (this machine's Tailscale device, forwarding its services to other tailnet devices only); public ingress without inbound ports.

3.4 Logical view: backend layers

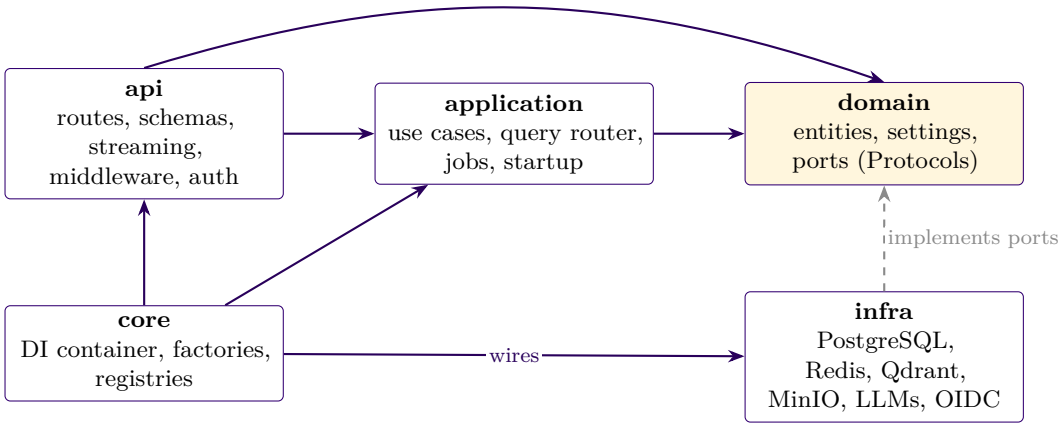


Figure 3.3: Backend layers; arrows show allowed dependencies

Table 3.2: Layer responsibilities

Layer	Path	Owns
Domain	`src/backend/domain/`	Entities (`Course`, `CourseTerm`, academic terms, WLU pages, retrieval plans, evidence and outcomes, jobs, principals), typed settings, ports. No framework or vendor code.
Application	`src/backend/application/`	Use cases (chat, scraping, ingestion, settings), the query router, job runners, startup workflows. Depends only on domain ports.
Infrastructure	`src/backend/infra/`	Adapters: SQLAlchemy/asyncpg repositories and dialect operations, Redis stores, Qdrant, MinIO, LLM and embedding clients, text splitters, the OIDC client.
API	`src/backend/api/`	HTTP routes, request/response schemas, NDJSON streaming, middleware, authentication and authorization dependencies, rate limiting.
Composition root	`src/backend/core/`	Dependency-injection container, factories and provider registries; the provision job and service-key tool.
Shared	`src/goldenguide_common/`	Configuration loading, secret reading, structured logging and telemetry, shared by the backend and the embedding service.

4 Design Decisions

Each record states the decision, the alternatives considered, the rationale and the consequences.

D1 Modular monolith rather than microservices

Decision: One backend codebase and process type. Separate containers only where the runtime profile genuinely differs: the embedding service and the data stores.
Alternatives: A service per capability (chat, ingestion, authentication).
Rationale: One codebase, one release cadence and one domain model. Microservices would add network failure modes, distributed transactions and deployment overhead with no scaling need behind them. Hexagonal boundaries keep a later split possible.
Consequences: Background jobs run in a separate worker container from the same codebase and image (decision D14).

D2 Structured retrieval first; semantic search where it fits

Decision: Factual questions are compiled into typed SQL queries. Vector search handles policies, services and open-ended questions. Both can contribute to one answer.
Alternatives: "Pure RAG": embed everything and retrieve the top chunks for every question.
Rationale: Chunk retrieval is probabilistic: it can miss the one section a student asked about, confuse terms, or return 8 of 30 sections. Prerequisites and section times must be exact. SQL answers exactly and cheaply; the LLM only chooses *which* lookups to run.
Consequences: More code (extraction, planning, compilation), justified by correctness on the most common questions.

D3 Deterministic extraction before the LLM; validated plans

Decision: Course codes, subjects, rooms and instructors are recognised in code against the database. The LLM plans only over masked entities, and its plan is validated before execution, with a conservative fallback plan.
Alternatives: Let the LLM read the raw question and emit queries or tool calls directly.
Rationale: Code is exact and free. Masking means the LLM cannot invent a course. Validation means a malformed or adversarial plan cannot trigger arbitrary queries.

D4 Hybrid vector search, cross-encoder reranking and whole-page expansion

Decision: Dense (MiniLM) and sparse (SPLADE) candidates fused in Qdrant, reordered by multi-vector (ColBERT) similarity, then a cross-encoder rerank, then whole pages when they fit the budget.
Alternatives: Dense-only search; a hosted search service.
Rationale: Dense vectors capture meaning but miss exact terms ("OSAP", course codes); sparse vectors catch terms; ColBERT adds token-level matching. A cross-encoder judges relevance rather than similarity. Whole pages fix "list everything" questions that chunking fragments. A retrieval evaluation measures each stage.

Consequences: A GPU is preferred; every stage is optional per retrieval profile. ColBERT only scores candidates the other searches found, so its per-token vectors carry no HNSW index: building one cost hours of indexing for a search that never used it.

D5 A separate embedding service

Decision: Embedding, sparse vectorisation and reranking run in their own FastAPI service.
Alternatives: Load models inside the backend; use only cloud embeddings.
Rationale: PyTorch/CUDA images are several gigabytes and need a GPU; the API should stay small, start fast and scale on CPU. The service can live on the one machine that has a GPU. Local models avoid per-token costs during full re-indexes. CPU and GPU builds are mutually exclusive dependency groups and can run side by side.

D6 Separated collection stages with raw sources kept

Decision: Discovery (crawling or downloading) stores raw HTML and LORIS extracts in object storage and records each URL's outcome; parsing into records is a separate job; indexing is a separate job again.
Alternatives: Parse and embed while crawling, in one pass.
Rationale: Each stage fails for different reasons (network, page structure, model). Keeping raw sources means a parser fix is re-run without crawling WLU again; re-indexing never re-downloads. Each stage reports its own outcome, so success of one is never reported as success of the pipeline.

D7 Ingestion as tracked, batched, idempotent jobs

Decision: Every collection and indexing step is a job with persisted run and batch state, `fresh`/`update`/`retry` modes, heartbeats and a zombie sweep.
Alternatives: A single ingestion script; a message queue and worker fleet.
Rationale: Crawls and re-indexes are long and fail partially. Persisted batches let `retry` replay exactly what failed; `update` processes only changed records. A durable queue was deferred: it adds infrastructure without a current need, and the job ports allow a worker later.
Consequences: Endpoints answer `202 Accepted` with a status URL to poll.

D8 Configuration-driven providers and profiles

Decision: Providers, models, retrieval profiles, chunking, enrichment and prompts are layered YAML, validated into typed settings and resolved through registries in the composition root.
Alternatives: Constants in code; environment variables for everything.
Rationale: Retrieval quality is tuned by experiment; changing a model or strategy should be a reviewed configuration change. Typed validation fails at startup rather than mid-request. Environment variables are reserved for deployment values and secrets.

D9 Redis for critical state with explicit failure policies

Decision: Sessions, single-use OAuth transactions and rate-limit windows live in Redis with append-only persistence; caches live there too.
Alternatives: Signed cookies (stateless sessions); in-process memory; PostgreSQL for sessions.

Rationale: Server-side sessions are revocable and keep tokens out of the browser; shared state lets every replica enforce the same limits.
Consequences: Two opposite policies by design: critical state **fails closed**, caches **fall back** to PostgreSQL.

D10 Provider-neutral OpenID Connect

Decision: Standard authorization-code flow with PKCE and nonce-verified ID tokens; everything provider-specific comes from the discovery document; roles must be explicitly mapped.
Alternatives: Keycloak-specific integration; custom username and password accounts.
Rationale: WLU students and staff already have Microsoft accounts: moving from Keycloak to Entra ID must be configuration. Explicit role mapping means a successful login never implies privileges. No custom cryptography or session mechanisms.

D11 NDJSON streaming with an explicit terminal event

Decision: Answers stream as newline-delimited JSON events ending in `ok` or `error`.
Alternatives: Server-Sent Events; WebSockets; one buffered response.
Rationale: A plain streamed POST passes through every proxy and Cloudflare without upgrade handling, carries typed events (text, citation, table, sources) and is trivial to parse. Once streaming starts the HTTP status cannot change, so completion must be signalled in-band.

D13 React single-page client on the same origin as the API

Decision: A React, TypeScript and Vite single-page application, built to static files and served by Nginx (or FastAPI) on the same origin as the API.
Alternatives: Next.js; a chat SDK such as Vercel AI SDK `useChat`; the previous plain HTML and JavaScript page.
Rationale: Same origin keeps the device cookie, server-side session and same-origin checks unchanged, and FastAPI remains the only backend. Next.js would add a Node server duplicating session and CSRF handling, and SEO does not matter for a chat app. Chat SDKs expect their own stream protocol. API types are generated from OpenAPI, so a contract change fails the frontend build.
Consequences: The NDJSON stream is outside OpenAPI, so the client validates each event at runtime; the runtime image must be rebuilt to ship a UI change.

D12 Docker Compose with profiles as the deployment unit

Decision: One Compose definition with a profile per service group; development and production overrides; one env file per environment (development, staging, production); CPU and memory limits on every container; loopback-bound ports; Tailscale between machines and a Cloudflare Tunnel for public ingress.
Alternatives: Kubernetes now; manual installation.
Rationale: The only prerequisite for a contributor is Docker, and the same definition runs on one PC or across machines. Kubernetes would be premature; the containers map directly to ECS or Kubernetes later.

D14 Background jobs on a Postgres-backed queue, run by separate workers

Decision: Jobs are stored as typed requests on their run and delivered through Procrastinate, an asynchronous job queue in the existing PostgreSQL, to `worker` containers with their own resource limits. The application's `job_runs`/`job_batches` remain the record of outcomes; Procrastinate only delivers work. In-process execution remains for development.
Alternatives: Keep jobs in the API process; Celery; Redis-backed async queues (arq, Taskiq); Temporal; a homegrown `SKIP LOCKED` queue.
Rationale: Jobs in the API process competed with chat for one event loop and core (status calls took seconds during imports) and died on every deploy. Procrastinate is asynchronous like the rest of the stack and needs no new infrastructure; Celery is synchronous-first, Redis is not the durable store, Temporal needs its own server and deterministic workflow code, and a homegrown queue would own every edge case. Keeping outcomes in `job_runs` preserves partial-success, batch and audit semantics the queue cannot express.
Consequences: A second PostgreSQL driver (psycopg) for the queue; Procrastinate upgrades need their SQL migrations wrapped in Alembic revisions; recording and queueing a run are two writes (risk R1).

5 Data Collection Subsystem

This chapter covers how source data moves from WLU into GoldenGuide: discovery, raw storage and parsing into structured records. Indexing for search is described in Chapter **??**.

5.1 Pipeline overview

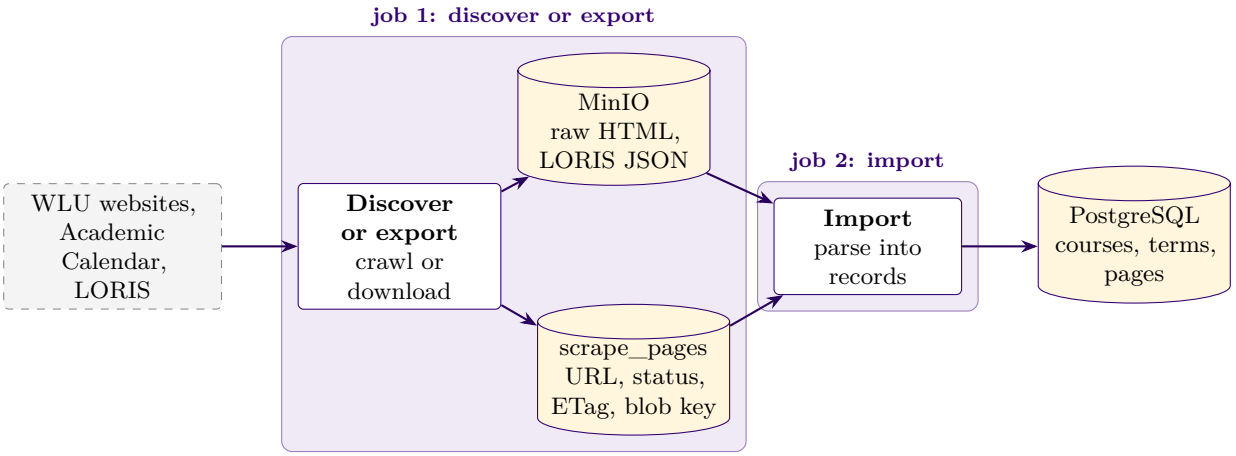


Figure 5.1: Collection: discovery and import are separate jobs

5.2 Sources

Table 5.1: Sources and their stages

Source	Discover / export	Import	Stored as
Academic Calendar	Reads the calendar home page to find that year's calendars, seeds the crawl with every page of their search results, and crawls only that year's URLs.	Classifies each page (course, program, glossary, ...) and extracts the course header and details.	`courses`, `wlu_pages`
WLU websites (`wlu`, `students_wlu`, `scholars_wlu`)	Breadth-first crawl seeded from sitemaps and seed pages, restricted to each site's allowed domains and excluded paths.	Extracts title and main content (configured selector, noise removed).	`wlu_pages`
LORIS	Downloads one term's sections from the Banner class search into one JSON object.	Links each section to its calendar course and upserts it.	`course_terms` and children

5.3 Crawler

The crawler (`application/services/simple_scraping/`) is a breadth-first asynchronous crawler built on `httpx`, driven by a *site profile* that decides which URLs are seeds, which may be followed and which are scraped.

- **Gate.** Every URL requested — seed, discovered link or redirect hop — passes the same gate: the profile's rules (allowed domains, excluded path patterns, ignored file extensions such as PDFs and images) and `robots.txt`.
- **robots.txt** is parsed per RFC 9309: the group matching the crawler's user agent applies (else `*`), groups for the same agent are merged, the longest matching rule wins. An unavailable `robots.txt` is treated as disallowing the host (risk R4).
- **Conditional requests.** In `update` mode stored pages are re-checked with `If-None-Match` and `If-Modified-Since` headers built from the stored validators; `304 Not Modified` keeps the stored copy.
- **Change detection.** HTML is uploaded to object storage only when its content hash changed.
- **Bounded concurrency** (four concurrent requests in the nightly run) and structured progress logs (start, milestones, heartbeat, finish).
- **Errors.** A network failure on one page is stored as retryable and the crawl continues; a failure in processing or persistence aborts the crawl and marks the run failed.

Table 5.2: Page statuses recorded in `scrape_pages`

Status	Meaning
`pending`	Discovered, not yet fetched
`success`	Fetched HTML that the profile scrapes; stored in object storage
`skipped`	Fetched only to discover links (not scraped)
`empty`	Fetched, but no usable content
`retryable_error`	Network failure; picked up by `retry` mode
`permanent_error`	Non-200 or non-HTML response; not parsed

Table 5.3: Crawl modes

Mode	Behaviour
`fresh`	Start from the profile's seeds and sitemaps; fetch everything (also finds new pages).
`update`	Re-check stored pages with conditional requests; only changed pages are stored again.
`retry`	Re-fetch pages that ended in a retryable error.

5.4 Raw source storage

Raw sources are kept in MinIO (S3-compatible) under deterministic keys, for example `academic-calendar/2026-2027/courses/<sha256-of-url>.html`. Calendar pages are versioned by academic year, so different years never collide. Each `scrape_pages` row records the URL, its version, status, HTTP status, content type, blob key, content hash, ETag, Last-Modified, retry count and timestamps (first seen, last attempted, last success), and a `needs_persistence` flag telling the import job which pages changed.

5.5 Parsing and import

- **Shared parsing** (`common_parsing.py`) converts HTML to clean Markdown using BeautifulSoup. Links in extracted text become numbered placeholders resolved when the text is shown, so stored text stays clean and links stay exact.
- **Academic Calendar import** parses a batch of stored pages and upserts them as courses or `wlu_pages` rows. Unknown subject codes *block the batch* (they must be added to the WLU lexicon) rather than producing unmapped records. Pages that cannot be parsed are logged, skipped and stay flagged for the next run.
- **Website import** extracts the title (repeated site suffix removed) and the main content region, and upserts `wlu_pages` rows.

- **LORIS export.** Banner keeps search state in the session, so the client resets it, selects the term and pages through the results; each section's prerequisites, corequisites and restrictions are fetched and the whole term is stored as one JSON object.
- **LORIS import.** Unknown subject codes stop the import before anything is written. Courses missing from the calendar are inserted *insert-only*, never overwriting calendar data. Each section is linked to its course for the governing academic year and upserted with its meetings and instructors. Sections whose course cannot be resolved are counted, not stored.
- **Audits.** Each crawl and import ends with an audit (page counts by status, record counts, a sample of unmapped pages) stored on the job run.

Provenance rule. The Academic Calendar is authoritative for course descriptions; LORIS for term sections. Credits and prerequisites can differ between them, so both are stored with their source. Imports distinguish insert-only, upsert, fill-missing and conditional updates.

6 Job Runtime

All collection and indexing work runs through one job model (`application/services/jobs/`).

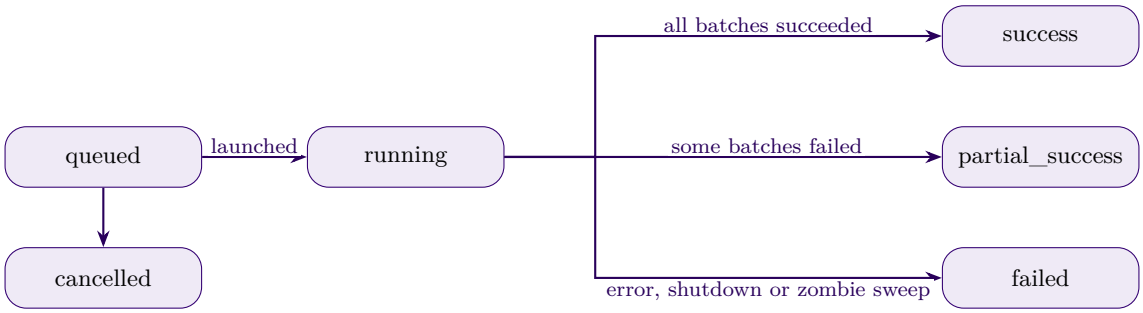


Figure 6.1: Job run lifecycle

- **Submission.** A run row is committed `queued` with its typed job request (job key and parameters), the launcher queues it, and the endpoint returns `202` with a status URL. Accepting a request is not completing it.
- **Execution.** In staging and production a `worker` container claims the run from a Procrastinate queue in PostgreSQL (only the run id is queued) and rebuilds the job from the stored request; in development the API runs it in-process. A run that is no longer queued is skipped, so a job never runs twice. CPU-bound parsing runs off the event loop: threads in the API, a process pool in the worker.
- **No overlap.** A second run of the same job name is refused (`409`) while one is queued or running, across all backend instances, using a PostgreSQL transaction advisory lock.
- **Batches.** Large jobs page through a table in id order (default 200 rows, up to three batches concurrently). A failed batch never aborts the others; it is stored `failed` with its error, and `retry` replays exactly those offsets.
- **Per-record failures.** One record failing does not stop its batch; the batch fails listing those records, which stay flagged for `retry` or `update`.
- **One-shot jobs** (a whole crawl, a LORIS export) use the same model with one synthetic batch, so every job shares one status and history view.
- **Heartbeats and recovery.** Running jobs send heartbeats. On shutdown, tasks are cancelled and awaited and interrupted runs recorded as failed. After a crash, a periodic zombie sweep (checked every two minutes) fails running runs without a heartbeat for five minutes and runs still queued after 24 hours; their unfinished batches become retryable. A periodic worker task releases queue entries of dead workers. Nothing is replayed automatically.
- **Scheduling.** A GitHub Actions workflow runs the full sequence nightly at 05:00 America/-Toronto with a scoped service key, waiting for each stage to succeed before the next.

7 Indexing Subsystem

Indexing turns structured records into searchable vectors. It reads PostgreSQL and writes Qdrant; it never touches the original sources.

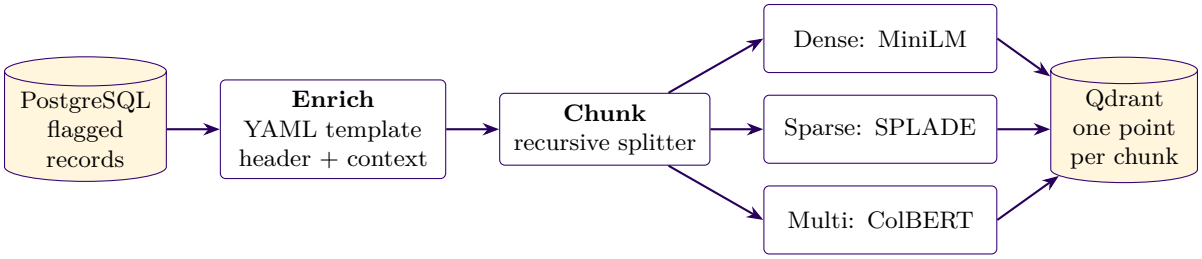


Figure 7.1: Indexing: record to vectors

7.1 Embedding job

One embedding source per table (courses, course terms, WLU pages, uploaded documents) defines how to page the table, build payloads and flag records. `fresh` embeds every record, `update` only records whose `needs_embedding` flag is set (set on import when the content hash changed), and `retry` re-runs the failed batches of the last run. The flag check happens in the job, not in SQL, so a batch offset always covers the same rows and `retry` replays exactly what failed.

7.2 Enrichment

A chunk on its own often lacks context: "Prerequisites: CP104" does not say which course it belongs to. Per record type (course, program, department, glossary, service page, ...) a template in `configs/catalogs/text_enrichment/retrieval_text.yml` renders a header with identifying facts, for example *Academic Calendar Course: CP264 — title; Subject; Academic Year; Prerequisites; Credits.* Strategies (`minimal`, `deep`, ...) trade token overhead for context and are chosen per vector collection.

7.3 Chunking

A recursive character splitter (LangChain) splits on paragraphs, then lines, then words. Presets range from 300 to 1,500 characters with 200 characters of overlap and are chosen per collection; every current collection uses the `large` preset (1,000 characters) with `deep` enrichment.

7.4 Embedding and storage

Each chunk is embedded with every vector type its collection defines and stored as one Qdrant point with named vectors and a payload (source table, source id, URL, title, chunk text). A record's old points are deleted *only after* all new vectors were generated, so a failure keeps the previous version searchable. Cloud embeddings (Gemini, OpenAI) are also supported per collection.

Table 7.1: Vector collection layouts

Layout	Vectors
`dense`	Dense only
`sparse`	Dense + sparse (hybrid)
`multi`	Dense + ColBERT multi-vector
`multi_sparse`	Dense + sparse + ColBERT multi-vector (current default)

Each collection carries a *version*. Changing its chunking, enrichment or vectors requires a new version and a re-index; provisioning refuses to run against an existing collection whose version does not match, so incompatible vectors are never mixed.

8 Embedding Service

The embedding service (`src/embedding_service/`) is a separate FastAPI application that serves every local model. It runs in one of two builds: **GPU** (CUDA PyTorch, ONNX Runtime GPU) or **CPU** (CPU-only PyTorch wheels). The builds are mutually exclusive dependency groups but separate containers can run side by side; the backend selects one with `EMBEDDING_RUNTIME` and the service host and port.

Table 8.1: Embedding service endpoints (all except health require an API key)

Endpoint	Purpose
`GET /health`	Liveness (used by the container healthcheck)
`POST /embed/query`	Dense or multi-vector embedding of a question
`POST /embed/documents`	Dense or multi-vector embeddings of chunks (batched, at most 16 texts per request)
`POST /sparse/query`, `/sparse/documents`	SPLADE sparse vectors
`POST /rerank`	Cross-encoder scores for (question, passage) pairs

Table 8.2: Local models

Role	Model	Library	Runtime
Dense embedding	`all-MiniLM-L6-v2`	sentence-transformers	GPU, CPU
Multi-vector	`colbert-ir/colbertv2.0`	FastEmbed (late interaction)	GPU
Sparse vectors	`prithivida/Splade_PP_en_v1`	FastEmbed (sparse)	GPU
Reranker	`BAAI/bge-reranker-base`	sentence-transformers cross-encoder	GPU
Reranker	`cross-encoder/ms-marco-MiniLM-L-6-v2`	sentence-transformers cross-encoder	CPU

- **Model loading.** The model registries are built when the service starts, and each model's constructor loads its weights, so every configured model for that runtime is downloaded into the `/models` volume (Hugging Face and FastEmbed caches) before requests are accepted. The healthcheck allows two minutes for the first start; later starts reuse the volume.
- **Registries.** Models are declared in `configs/catalogs/providers/ml\_local.yml` and mapped to implementations (MiniLM, FastEmbed ColBERT, SPLADE, cross-encoder) by a registry; adding a model type is one class and one registry entry.

- **Limits.** Thread counts are bounded (`EMBEDDING_THREADS`); the backend client uses a 180-second timeout and batches documents.
- **Telemetry.** FastAPI instrumentation exports traces (health checks excluded); logs share the backend's JSON format.

9 Question Answering Subsystem

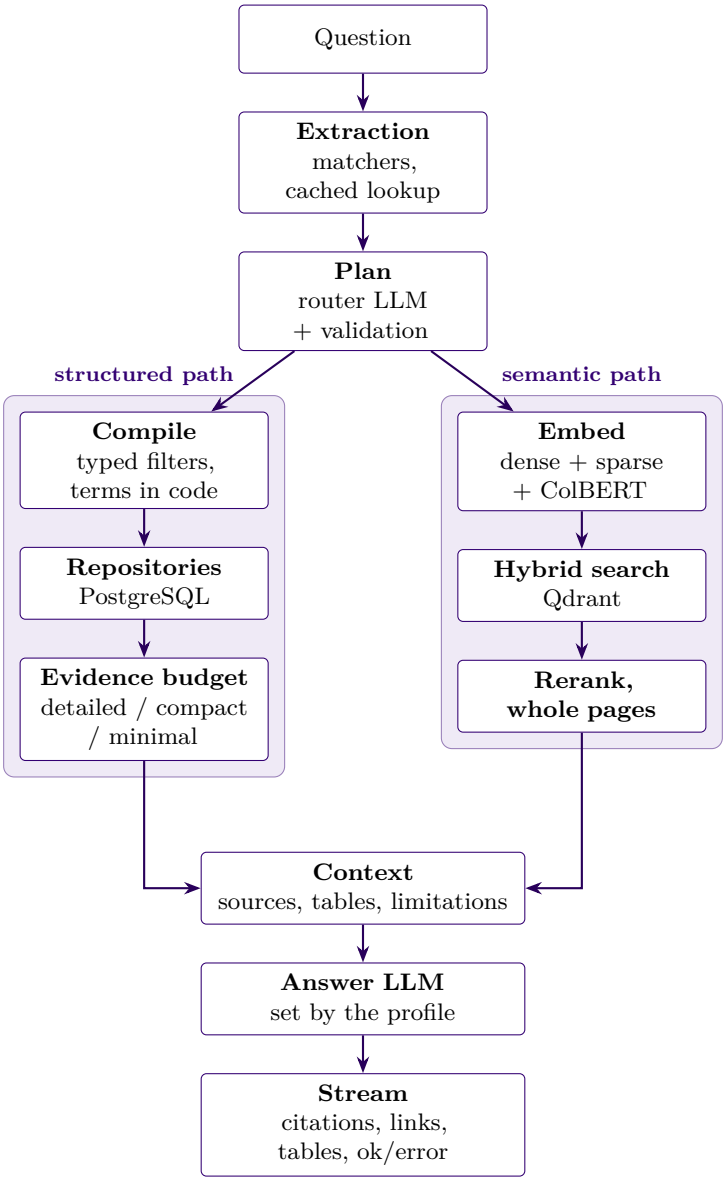


Figure 9.1: Question answering pipeline

9.1 Retrieval profile

A retrieval profile bundles the vector collection, search strategy, structured-evidence budget and answer LLM, and may name fallback profiles that semantic search tries if it fails. Every question currently uses `deep_cloud` (`multi_sparse` collection, hybrid search reordered by ColBERT to 60 candidates, rerank to 20, whole pages up to 30,000 tokens). Its answer LLM is Gemini, and semantic

search falls back to `balanced_cloud`. The settings also accept fallback answer models, but none are configured.

9.2 Deterministic entity extraction

No LLM is used. Matchers check the question against an *entity lookup* of every known course, subject, program, room and instructor, built from the database and cached in Redis for a day (on a cache outage the process uses its last copy or PostgreSQL). Recognised entities are masked (`[COURSE1]`, `[DEPARTMENT1]`) before planning.

Table 9.1: Entity matchers

Matcher	Finds
Course code	`CP264`, `cp 264` and lexicon aliases (`comp sci 264`), kept only if the course exists
Lexicon	Faculties, departments and subjects by exact name or WLU alias (e.g. "fhss")
Department hint	Misspelled or truncated department names (fuzzy); passed to the planner, never masked
Room	LORIS room identifiers as students write them
Instructor	Full names of instructors teaching LORIS sections

9.3 Query planning

The router LLM turns the masked question into a retrieval plan: requests with a *source* (`courses`, `course_terms`, `wlu_pages`, `vector`), an *operation* (`lookup`, `filter`, `related`, `semantic`), entity masks and filters (term, credit, level, days, rooms, instructors). LLM output enters the domain in one place and is:

1. **leniently parsed**, repairing only shapes LLMs commonly get wrong (a single value where a list is expected), driven by the domain model's types;
2. **validated**: each operation must be supported by its source and every mask must be one the extractor produced;
3. **replaced by a fallback plan** if rejected: look up mentioned courses, otherwise semantic search.

9.4 Structured path

1. **Enrichment.** Every mentioned course receives a course lookup and a current-term offering lookup, even if the planner omitted one. Duplicate requests run once.

2. **Compilation.** Course and course-term compilers turn each request into typed repository filters, one per (term, course or subject). Terms are resolved by the academic term resolver, never by the LLM. A filter the database cannot execute is reported as *unsupported*.
3. **Repositories** query PostgreSQL through domain ports, fetching one record beyond the limit to prove that more exist ("results limited").
4. **Evidence budget.** Each record has detailed, compact and minimal renderings; the whole set uses the richest tier the profile allows. If even the minimal tier does not fit, the request is reported as unsupported; records are never silently dropped.
5. **Tables.** LORIS sections and planned listings become tables built from the exact records; the model places a table by writing `{{TABLE:n}}` instead of retyping rows.

9.5 Semantic path

Runs when the plan requests it, or as a fallback when the structured path found nothing and the plan allows it: embed the question with every vector type of the profile's collection, run hybrid search in Qdrant (dense and sparse candidates, each filtered, fused with Reciprocal Rank Fusion, then reordered by ColBERT), rerank with the cross-encoder, drop duplicate chunks, and replace chunks with whole pages when the reranked pages fit the budget together. A failing provider moves to the next profile in the profile's fallback list; if semantic search fails entirely, the answer is told so.

9.6 Context assembly and generation

- Structured evidence first, then semantic results; each source is one numbered block with one citation entry. The prompt asks for a `[n]` citation on every fact, even with a single source, and repeats it right after the context.
- URLs are replaced by short placeholders that the model copies back and the stream restores, so the model never retypes or invents a link.
- Every request has an outcome — completed (with count and truncation), unsupported or failed — and the context tells the answer model which parts could not be answered and why.
- Today's date and the terms in scope are computed in code and given to the prompt.
- The answer is streamed through a chain applied to every chunk: citations (`[n]`, `[n, m]` or `[Source n]` → citation event) → links (placeholder → URL) → tables (marker → table event). Each stage holds back partial markers across chunk boundaries and releases them in order.

Table 9.2: Retrieval outcomes and what the student sees

Outcome	Meaning	Student sees
Completed, no records	The lookup ran and found nothing	"No records found" for that part
Unsupported	The plan asked for something not yet queryable	That part cannot be answered
Failed	A provider or the database failed	That part could not be checked right now
Truncated	Retrieval hit a limit	Results are partial; more exist

9.7 Conversations and follow-up questions

- Every turn is stored: the question before answering, the answer however the stream ends (complete, error or interrupted), in a shielded save that a disconnect cannot cancel. A complete answer is saved before the stream ends and its message ID is sent in a `saved` event, so the student can rate it at once. Assistant messages keep diagnostics: query plan, retrieval outcomes, sources, profile, settings version, timings, `trace_id` and `request_id`.
- Storing history is best-effort: if it fails the answer still streams and the failure is logged.
- Follow-ups: the last completed turns (bounded by `configs/catalogs/chat/chat.yml`) are sent to the answer model for meaning only; retrieval runs again and citations always come from the current question's evidence. The previous answer's entities are offered to the router under `[EARLIER_...]` masks, so "is it offered in winter?" resolves deterministically; plan validation rejects any mask outside the current and carried sets.
- Each answer also stores a versioned snapshot of its displayed blocks (text, citations, tables), so history renders exactly as the live answer did.
- Ownership is the anonymous `gg_client` device cookie, stored as a SHA-256 hash. Retention is not yet built.

10 Web Client

The web client in `frontend/` is a React single-page application (decision D13). It holds no academic logic: it sends questions, renders the stream and shows stored conversations.

Table 10.1: Web client stack

Concern	Choice
Application	React 19, TypeScript (strict), Vite
Routing and server state	TanStack Router (`/`, `/c/$conversationId`); TanStack Query
API types	Generated from the backend's OpenAPI document; `openapi-fetch`
Stream validation	Zod schema per event
Interface	Tailwind CSS, shadcn/ui (Radix), `react-markdown` without raw HTML
Tests	Vitest, Testing Library, Mock Service Worker, Playwright

10.1 Answer streaming

- A reader decodes the response incrementally and parses complete lines only, so lines and multi-byte characters split across network chunks are handled.
- Each line is validated. Unknown event types are skipped so additive backend changes do not break deployed clients; a known type with the wrong shape is a protocol error, never rendered.
- A pure state machine turns events into an answer with one of six phases: pending, streaming, complete (last event `ok`), error, interrupted (no terminal event) and stopped (the student pressed Stop). Partial text is kept but always marked incomplete.
- Tables are rendered from `table` event data, never through Markdown, so scraped characters cannot change their structure. Citations become inline source chips with a preview card (site, title, snippet).

10.2 Conversations in the client

- The live chat session sits above the routes, so a new chat's first answer keeps streaming when the URL changes to the conversation returned in `X-Conversation-Id`.
- Stored answers are replayed from their snapshots through the same state machine and renderers as live answers; older messages fall back to their text.
- Status events carry codes only; the client owns all student-facing wording, including the name of the source being searched ("Searching LORIS registration…").

- Every completed answer offers thumbs up/down (with a reason for thumbs down) as soon as the `saved` event gives its message ID; stored answers offer it too.

10.3 Serving

Table 10.2: Ways the client is served

Mode	How
Runtime image	A Node stage builds the app; Nginx serves it with a client-route fallback, `index.html` uncached and hashed assets cached as immutable, and proxies `/api` unbuffered
Docker development	Nginx proxies the UI and hot-reload socket to an internal Vite service; source is bind-mounted
FastAPI only	`SERVE_FRONTEND=true` serves the built files with the same fallback; startup fails if the build is missing

11 Cross-cutting Concerns

11.1 Authentication and authorization

Table 11.1: Access policy

Caller	Can do
Anyone	Chat, rate-limited per client
Signed-in student	Same as anyone today; a student-only policy exists if WLU requires sign-in for chat. The student role is mapped only when the identity provider defines it; the bundled Keycloak realms define only the administrator role
Signed-in administrator	Every administrative route and the API documentation; holds every scope
Service client (bearer key)	Only its configured scopes: `sources:collect`, `indexing:sync`, `jobs:read`, `settings:write`, `documents:write`
This machine, sign-in disabled, `LOCAL_ADMIN` on	Administrator, for development only; never through the tunnel, never in production

- **Sign-in** uses the OpenID Connect authorization-code flow with PKCE (S256) and a nonce-bound ID token (Authlib, joserfc). The ID token's signature, issuer, audience, expiry and nonce are verified. Only explicitly configured roles count; a successful login alone grants nothing.
- **Sessions** are server-side in Redis; the browser holds only an opaque random cookie (`HttpOnly`, `SameSite=Lax`). Subjects are issuer-qualified.
- **Keycloak.** Development imports `deploy/keycloak/realm.json`, whose one user comes from `KC_TEST_ADMIN_USERNAME`, `KC_TEST_ADMIN_PASSWORD` and `KC_TEST_ADMIN_ROLE`. Staging and production each run their own Keycloak at their own HTTPS hostname (`KC_HOSTNAME`) with `deploy/keycloak/realm.hosted.json`: redirect URIs from `SITE_ORIGIN`, PKCE required, brute-force protection, no self-registration, and one seeded user, the first administrator (`KC_ADMIN_USERNAME`, `KC_ADMIN_PASSWORD`, `KC_ADMIN_ROLE`). The realm is imported only into an empty volume. The Keycloak console account (`KC_BOOTSTRAP_ADMIN_USERNAME`) is separate from GoldenGuide users. Keycloak still runs in its development mode (risk R13).
- **Microsoft Entra ID** is a configuration change (discovery URL, client, app roles), because no layer is Keycloak-specific.
- **Service keys** are stored only as SHA-256 hashes, two per client for rotation; a bearer request never falls back to the cookie.
- **Local administrator** (development): with sign-in disabled and `LOCAL_ADMIN` enabled, requests that did not pass through the Cloudflare tunnel (no `CF-Connecting-IP`) and come

from a loopback or private peer are treated as an administrator. This relies on ports being bound to loopback; the backend refuses to start with it in production.

- **Fail closed.** No session store means no access; a missing credential stops production from starting; sign-in disabled without local admin returns `503`.
- **Hardening.** Same-origin checks on state-changing requests, a content security policy with configurable frame ancestors (the API documentation's inline initializer is allowed by its SHA-256 hash on that response only, never by allowing inline scripts), trusted hosts, an error boundary returning one envelope without internals, and per-client rate limits stored in Redis (forwarded addresses trusted only from configured proxies).

11.2 Configuration

Behaviour is versioned YAML; deployment values and secrets are environment variables. `ConfigLoader` merges `configs/base/`, then `configs/catalogs/<group>/`, then `configs/environments/<APP_ENV>.yml` (later wins), resolves references between entries and `${secrets:NAME}` values through the `SecretReader` port, and validates the result into typed Pydantic settings at startup. Catalogs follow one pattern: *providers* (things that exist), *profiles* (reusable combinations), *active* (what this deployment uses). Some settings (prompts, model choices) can be changed at runtime by administrators; they are versioned in PostgreSQL and cached in Redis. Appendix **??** lists every configuration file.

11.3 Observability

- **Logs.** Every service logs one JSON object per line (structlog) with `request_id` and `trace_id`; token-like fields are redacted. Alloy ships container logs to Loki with low-cardinality labels (service, container, level, host); `trace_id` is structured metadata.
- **Traces.** OpenTelemetry instrumentation (FastAPI, httpx, SQLAlchemy, explicit spans) exports through the Collector to Jaeger.
- **Grafana** links each log line to its trace and each span to its logs.
- **Timings.** Each answer logs routing, structured, vector-search, first-chunk, generation and total times. Every error response carries `request_id` and `trace_id`.

12 Data Design

12.1 Relational model

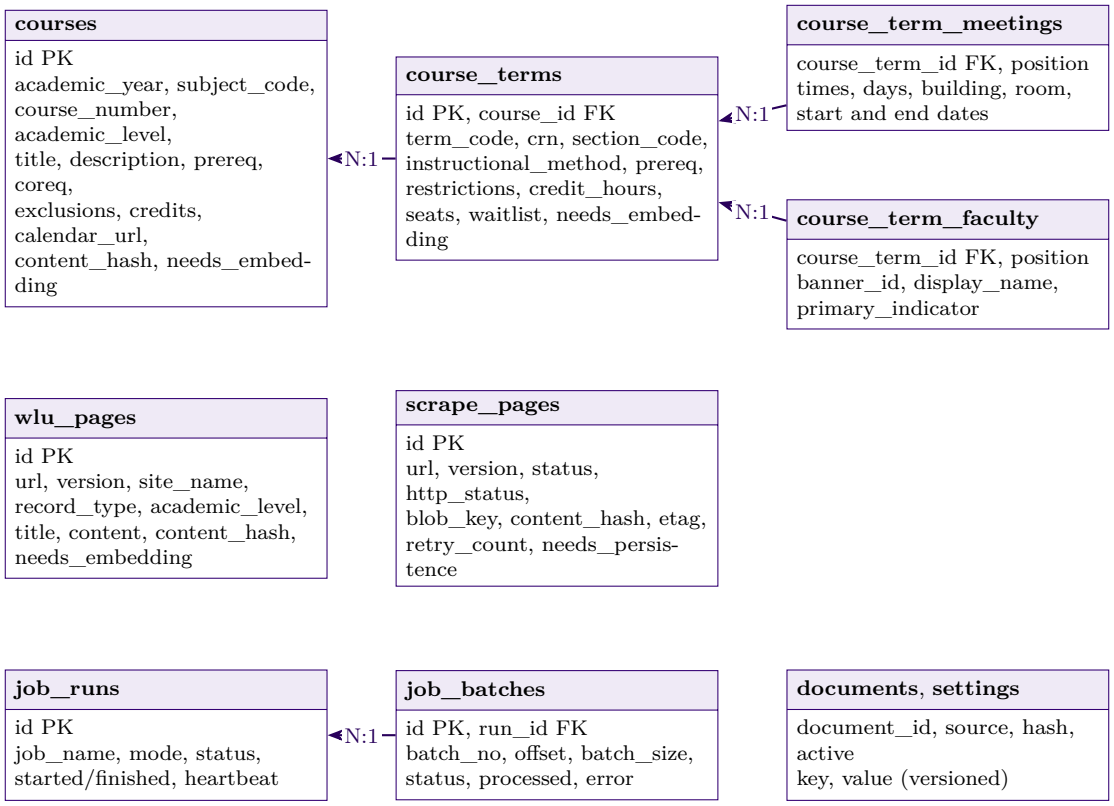


Figure 12.1: PostgreSQL entities (arrows point from child to parent)

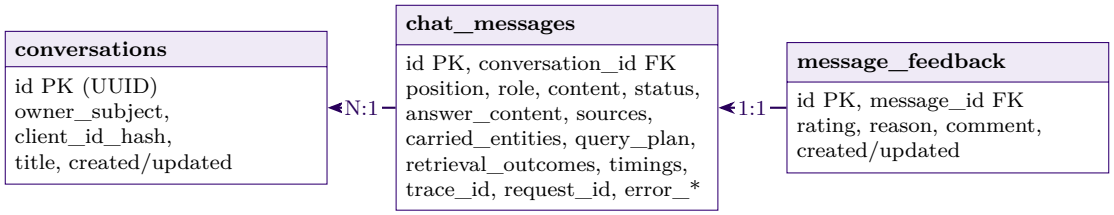


Figure 12.2: Conversation entities (deleting a conversation cascades to its messages and feedback)

Table 12.1: Natural keys

Table	Unique key	Why
`courses`	academic year, institution, subject code, course number	A course is stable per calendar year (`Course` identity)
`course_terms`	term code, CRN	A LORIS section is unique within its term (`CourseTerm` offering)
`wlu_pages`, `scrape_pages`	URL, version	The same page in different calendar years is a different record
`job_batches`	run, batch number	A batch is replayed by position, never duplicated
`chat_messages`	conversation, position	Turns keep a strict order within a conversation
`message_feedback`	message	One rating per answer; rating again replaces it

- **Course versus course term.** `courses` describes the calendar entry; `course_terms` a term's sections, which can carry different credits, prerequisites and restrictions. Both keep their own copy with their source.
- **Academic level.** `courses` and `wlu_pages` carry `academic_level` (`undergraduate` or `graduate`), the Academic Calendar a record comes from. It is `NULL` for records that apply to every student (tuition, important dates, website pages) and for courses known only from LORIS. A level filter returns that level plus the `NULL` records. Course terms take the level of their course.
- **Change detection.** `content_hash` detects real changes on import; `needs_embedding` flags records for indexing and stays set when embedding fails.
- **Schema evolution** happens only through Alembic migrations applied by the provision job; the API refuses to serve an outdated schema; every migration must downgrade (tested). The pre-release migrations were folded into one baseline revision (`0001`); existing databases whose schema matched it were stamped to it without changing data.

12.2 Vector, object and ephemeral data

Table 12.2: Non-relational stores

Store	Contents	Failure policy
Qdrant	One point per chunk: named vectors (dense, sparse, ColBERT) and payload (source, id, URL, title, academic level, text). Derived data, rebuildable from PostgreSQL; versioned collections. A payload-only indexing run rewrites payload fields without embedding again.	Retrieval reports the semantic part as failed; the answer says so
MinIO	Raw calendar and website HTML (versioned keys), LORIS term extracts, uploaded originals.	Import jobs fail and are retried
Redis	Sessions; single-use OAuth transactions; rate-limit windows; caches for the entity lookup, settings and courses.	Critical state fails closed; caches fall back to PostgreSQL

13 Interface Design

13.1 HTTP API

All routes are under `/api/v1`; administrators can browse OpenAPI documentation at `/docs`.

Table 13.1: HTTP endpoints

Method and path	Access	Purpose
`POST /chat/stream`	Public, rate-limited	Ask a question; NDJSON stream; conversation ID in `X-Conversation-Id`
`GET /chat/conversations`	Device owner, rate-limited	This device's conversations
`GET /chat/conversations/\{id\}/messages`	Device owner, rate-limited	Stored turns with answer snapshots and sources
`DELETE /chat/conversations/\{id\}`	Device owner, same origin	Delete a conversation
`PUT /chat/messages/\{id\}/feedback`	Device owner, same origin	Rate an answer, with an optional reason
`GET /auth/login`, `/auth/callback`	Public	OpenID Connect sign-in
`GET /auth/me`, `POST /auth/logout`	Signed in	Current principal; end session
`GET`, `PATCH /admin/settings/`	settings:write	Runtime defaults (versioned)
`GET /admin/documents/`, `POST .../upload`	documents:write	List and upload documents
`POST /admin/sources/academic-calendar/{discover,import}`	sources:collect	Calendar crawl; parse
`POST /admin/sources/websites/{discover,import}`	sources:collect	Website crawl; parse
`POST /admin/sources/registration/{export,import}`	sources:collect	LORIS download; load
`POST /admin/indexing/{courses,course-terms,pages}/sync`	indexing:sync	Build or update vectors; `payload_only` rewrites their payload without embedding
`GET /admin/jobs/{run_id}`	jobs:read	Job status, batches, outcome
`GET /health/live`	Public	Liveness

13.2 Chat stream contract

Table 13.2: Stream events

Event	Meaning
`{"type": "status", "stage": ..., "source": ...}`	Progress before the text, not terminal: `understanding`, `searching` (with the source from the plan) or `writing`; codes only
`{"type": "text", "text": ...}`	Answer text, in order
`{"type": "citation", ...}`	Citation marker with source numbers and URLs
`{"type": "table", ...}`	Table of database records, with title, columns and rows
`{"type": "sources", ...}`	Source list (title, URL, type, preview), sent once before the answer text so citations can show their sources while streaming
`{"type": "saved", "message_id": ...}`	The complete answer was stored under this ID (used to rate it); just before `ok`, omitted if history could not be saved
`{"type": "ok"}`	Terminal: the answer is complete
`{"type": "error", ...}`	Terminal: failed after the stream started; same envelope as HTTP errors

An answer is complete only if the last event is `ok`; a stream ending without `ok` or `error` is incomplete. Errors share one envelope: code, message, `request_id`, `trace_id`.

13.3 Internal interfaces

- **Domain ports** (`src/backend/domain/ports/`): repositories, vector store, blob storage, caches and stores, LLM, embedding, reranker, sparse vectorizer, text splitter, job repository, secret reader. Adapters for vector stores and caches must pass shared contract test suites.
- **Embedding service API** (Table **??**), protected by an API key.
- **Configuration** (Appendix **??**) and **telemetry** (OTLP, JSON logs).

14 Interaction Design

14.1 Answering a question

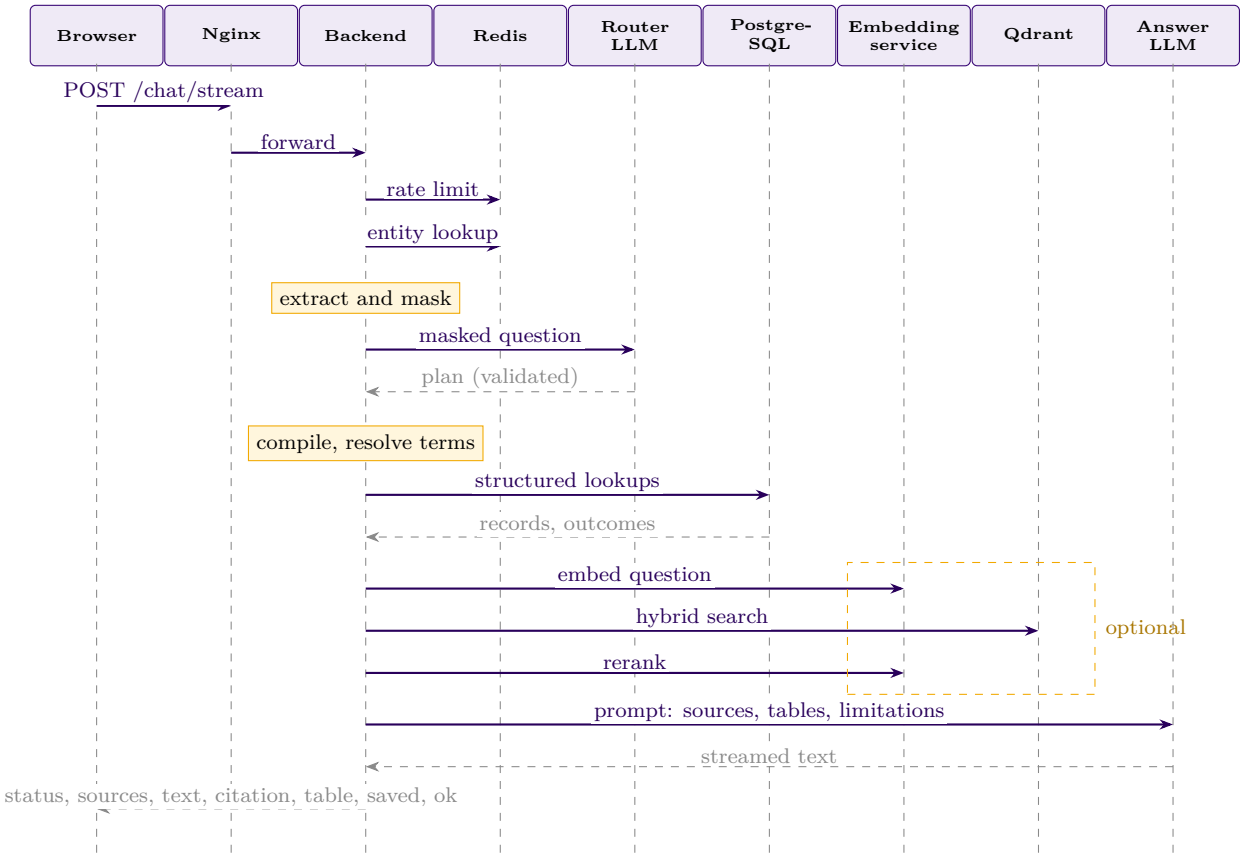


Figure 14.1: Sequence: answering a question

14.2 Signing in

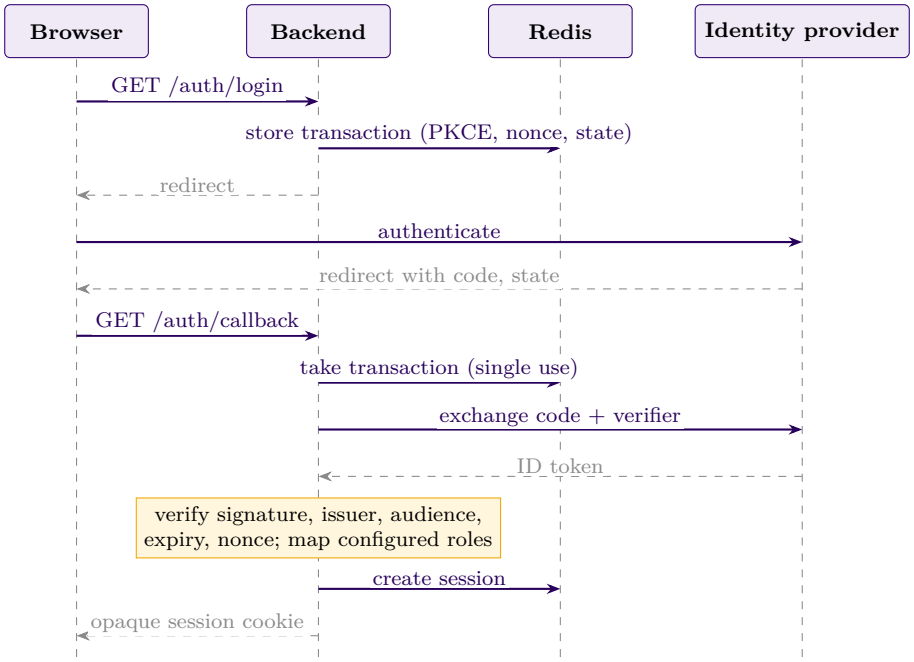


Figure 14.2: Sequence: OpenID Connect sign-in

14.3 Scheduled ingestion

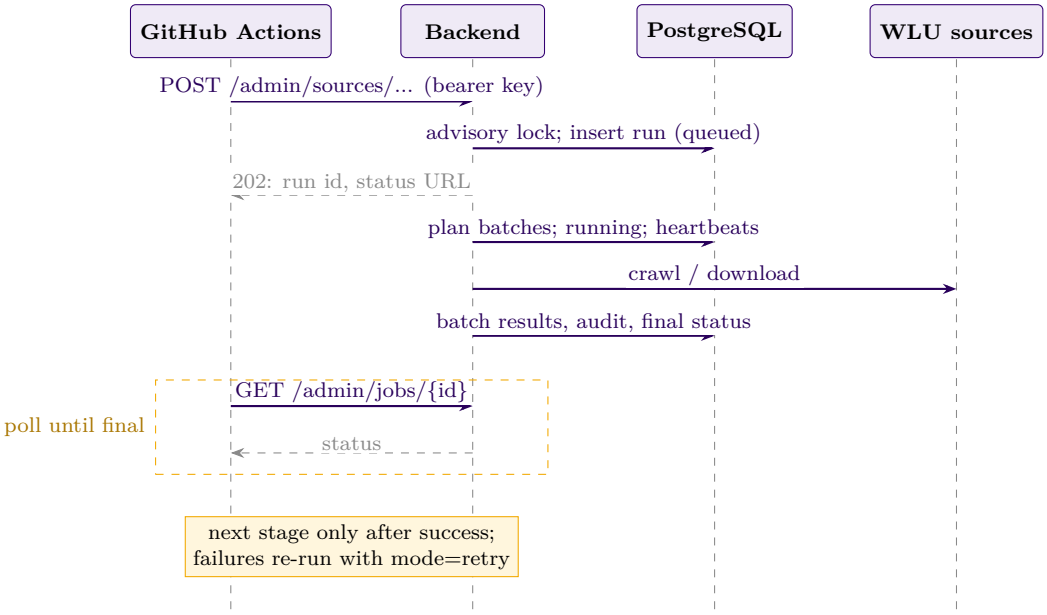


Figure 14.3: Sequence: a scheduled ingestion stage

15 Quality Attributes

Table 15.1: How each non-functional requirement is met

Requirement	Mechanisms
N1 Trustworthiness	Evidence-only prompts; masked planning; validated plans; link placeholders; tables built from records; limitation notes
N2 Provenance	Separate course and course-term records; insert-only calendar protection; natural keys; raw sources kept; citations on every source
N3 Reliability	Per-request outcomes; profile fallback chains; batch-level retry; vectors replaced only after new ones exist; heartbeats and zombie sweep; separated collection stages
N4 Security	OIDC with PKCE and nonce; server-side sessions; explicit roles and scopes; hashed service keys; same-origin checks; security headers; fail-closed state; loopback-bound ports; required production secrets
N5 Scalability	Stateless API with shared Redis state; replica-safe job locks; per-service profiles; independently scaled embedding service
N6 Observability	JSON logs, OpenTelemetry traces, Grafana correlation, identifiers in every error, per-answer timings, job audits
N7 Maintainability	Hexagonal layers; dependency injection; registries; typed settings; behavioural unit tests and adapter contract tests
N8 Operability	Docker-only setup; working default environment; resource limits; locked image versions and lockfile; provision job
N9 Cost control	No LLM for extraction or term resolution; evidence token budgets; rate-limited chat; local embeddings; cheaper profiles available
N10 Politeness	RFC 9309 robots.txt handling; conditional requests; bounded concurrency; excluded paths and file types

16 Deployment and Operations

16.1 Environments

Table 16.1: Environments

Environment	Shape
Development	One machine by default, configured by `.env`: base Compose file plus development overrides (source mounts, auto-reload, shared development virtual environment, IDE workspace, test database), full observability; Keycloak only when sign-in is tested. It may instead be split across machines by role over Tailscale, one env file per machine (Section **??**). A developer may expose it through their own Cloudflare Tunnel.
Staging	A production-style stack for the team (QA, staff), configured by `.env.staging`: the production Compose overrides with its own data stores, credentials, OIDC client and Cloudflare Tunnel at staging.askgoldenguide.ca, intended to be limited to the team by a Cloudflare Access policy. Today the backend's production-only startup checks (HTTPS origin, no `LOCAL_ADMIN`, required trace exporter) apply only when `APP_ENV` is `production` (risk R12).
Current production	Base Compose file plus production overrides (all credentials required), configured by `.env.prod`, on one self-hosted machine; public access through a Cloudflare Tunnel to askgoldenguide.ca; optional additional machines over Tailscale.
Planned cloud	ECS/Fargate for the API and provision job; a GPU instance for embeddings; RDS (PostgreSQL), ElastiCache (Redis), S3 (objects), managed or self-hosted Qdrant; Secrets Manager behind `SecretReader`; Entra ID for sign-in.

Each environment has its own env file, selected with `docker compose -env-file`; the file also sets `ENV_FILE` so the containers read the same values, and `COMPOSE_FILE` and `COMPOSE_PROFILES` so commands need no `-f` or `-profile` flags (`docker compose up -d`, `docker compose -env-file .env.prod up -d`). Only the `.env.dev.example`, `.env.staging.example` and `.env.prod.example` templates are committed. The staging and production templates leave every credential blank, so a deployment cannot start with default passwords. In a larger deployment the same values are injected as environment variables by a secret manager or the deployment pipeline instead of a file. Each env file also sets `COMPOSE_PROJECT_NAME` (`goldenguide-staging`, `goldenguide-prod`; development keeps `goldenguide`), so containers, networks and volumes are separate per environment even on one machine. Staging and production each expose their own Keycloak hostname (`auth-staging.askgoldenguide.ca`, `auth.askgoldenguide.ca`) through their tunnel; the administration console paths are limited to the team by a Cloudflare Access policy.

16.2 Compose profiles and images

Profiles select service groups (`database`, `vector-store`, `object-store`, `state_store`, `provision`, `backend`, `frontend`, `auth`, `observability`, `log-shipper`, `tailnet`, `tunnel`); the aggregates `all` includes everything except the embedding service, `all-dev` also leaves out the tunnel and Keycloak (sign-in is off in development; `auth` adds Keycloak) and adds the IDE workspace, and `services` holds the shared data stores and Keycloak for a machine that serves them to others. `tailnet` is in no aggregate, so a single machine never joins a network. The embedding service is chosen explicitly with `embedding-gpu` or `embedding-cpu` (or neither when another machine runs it). One multi-stage Dockerfile builds every Python image: a non-root base, a toolchain with the locked uv version, a development stage, a builder that installs one dependency group with `uv sync -locked`, and a minimal runtime stage without compilers.

16.3 Multiple machines

Docker is the only requirement for one machine. Splitting the stack across machines is optional and uses Tailscale, a private WireGuard network: each machine runs only its own profiles, for example a *data node* with the data stores, provisioning and GPU embeddings, and an *application node* with the backend, frontend and observability. Each machine has its own env file (`ENV_FILE`, `MACHINE_NAME`, its profiles plus `tailnet`), and names the others once, e.g. `DATA_NODE_HOST`, which client settings reference (`DB__POSTGRES__HOST="${DATA_NODE_HOST}"`); no IP addresses are configured.

- **Incoming.** The `tailscale` container is its own tailnet device, `<MACHINE_NAME>-docker`, on the Compose networks. It forwards a fixed list of ports (`deploy/tailscale/serve.json`) to services by name, so only devices on the tailnet reach them. Nothing extra is published on the host: every published port stays on loopback, which keeps local-admin requests (Section **??**) limited to the machine itself. It runs in userspace mode, so it needs no extra privileges, and works the same on Docker Desktop and Linux.
- **Outgoing.** Containers that call another machine (the backend, and logs and traces sent to the observability machine) reach tailnet names through the host's Tailscale client today. A gateway container sharing the `tailscale` container's network would make Docker the only install on every machine; it is future work.
- **Observability.** One machine runs the `observability` profile; the others run `log-shipper` and send logs and traces to it. `MACHINE_NAME` labels every log line, so Grafana shows all machines together.

This was verified with the data node on one machine and the application node on another. Every forwarded port is open to the whole tailnet, so a shared tailnet needs Tailscale access rules; and PostgreSQL and Redis rely on the tailnet for encryption (risk R5). Production replaces this with managed endpoints on a private network and a secret manager (Table **??**, planned cloud).

16.4 Resources and storage

Every container has CPU and memory limits with swap disabled. Large dependencies are stored once: runtime images carry each service's environment and are reused by development for the embedding services; the development environment shares one volume between the uv cache and the virtual environment so packages are hard-linked rather than copied. Model weights live in their own volumes. Images that declare volumes receive a temporary file system so no anonymous volumes are created. Qdrant and MinIO use locked image versions (pinned by digest: an exact image fingerprint rather than a movable tag such as `latest`), so an update can never silently change how they store data on disk; upgrading them is a deliberate step.

17 Verification

Table 17.1: Test strategy

Kind	Needs	Covers
Unit	Nothing (fakes at domain ports)	Extraction, term resolution, plan validation, compilers, evidence budgeting, streaming, authorization decisions, use cases
Integration	Real PostgreSQL, Redis, Qdrant	SQL filters and upserts, migrations (round trips), transactions, Redis atomicity and TTLs, Qdrant search; shared adapter contract suites
Evaluation	Running stack and data	Whether each benchmark question's answering page reaches the answer model, before and after reranking, with timings; end-to-end answers on demand
Frontend	Node only (API mocked)	Stream parsing and answer phases, history replay, chat page and composer, tables, sources, copy, theme (Vitest, Mock Service Worker); the production build in desktop and phone viewports (Playwright)

Integration tests use an isolated `goldenguide_test` database (names must end in `_test`), a separate Redis database with random key prefixes and uniquely named Qdrant collections, so development data is never touched. A missing test service skips locally and fails in CI. GitHub Actions runs every unit and integration test on each pull request against disposable service containers, without application secrets or paid LLM calls; a branch ruleset makes the check mandatory.

18 Risks, Limitations and Roadmap

Table 18.1: Risks and technical debt

ID	Item	Impact	Mitigation or plan
R1	A worker restart interrupts its running job; jobs are not retried automatically	A long import must be resubmitted	Graceful shutdown (60 s) records interrupted runs; imports are idempotent with `update`; a run recorded but not queued (API crash between the two writes) is failed after 24 hours
R2	WLU page structure can change	Parsers extract less	Audits record counts; nightly runs surface failures; raw HTML allows re-parsing
R3	LLM outages or behaviour changes	Degraded answers	Plan validation; explicit error events; evaluation before changes. Retrieval profiles fall back on search failures; answer-model fallback is supported but not configured
R4	An unavailable robots.txt disallows the host	A crawl can succeed with no pages	Logged; check audits, not only statuses
R5	PostgreSQL and Redis connections are not encrypted	Exposure on untrusted networks	Keep them on loopback, private networks or Tailscale
R6	Layer rules enforced by review, not tooling	Boundaries can erode	Add import-linter contracts to CI
R7	Every question uses the deepest profile	Higher cost and latency	Route simple questions to cheaper profiles
R8	No readiness endpoint	Orchestrators cannot detect broken dependencies	Add a dependency-aware readiness check
R9	Questions and answers are logged	Student data in logs	Treat retention and access as student data
R10	Chat is public	Abuse and cost	Redis-backed rate limits; student-only policy available
R11	Local administrator relies on loopback-bound ports	Exposure if ports are published widely	Disabled by default; refused in production; documented

ID	Item	Impact	Mitigation or plan
R12	Staging does not enforce the production-only startup checks	Staging could run with an HTTP origin or `LOCAL_ADMIN` that production would refuse	Apply the checks to staging as well; the staging template sets safe values
R13	Staging and production Keycloak run in development mode with its built-in database	Limited durability and hardening of the identity provider	Suitable for a small team; move to production mode with PostgreSQL and backups, or to WLU's Entra ID
R14	Conversations are kept with no retention period	Student data accumulates	Add a retention job; deleting a conversation is already available

18.1 Roadmap

1. Microsoft Entra ID sign-in for students and faculty, with a faculty role.
2. Course syllabi and faculty documents as sources.
3. Routing simple questions to cheaper retrieval profiles.
4. AWS deployment, and a secret manager in place of env files for staging and production.
5. A Docker-only Tailscale gateway for outgoing connections, so no machine in a multi-machine setup needs the Tailscale client installed.
6. Replicas with load balancing across machines (for example several GPU embedding machines behind one name) through an orchestrator: Docker Swarm as the smaller step from Compose, Kubernetes for autoscaling and GPU scheduling. Both need Linux hosts and an image registry.
7. Readiness endpoint and a durable job worker.
8. Web client: an evidence panel showing the cited page and excerpt, a term-aware prerequisite graph, and distinct screens for each retrieval outcome.
9. Import-linter contracts and broader evaluation coverage in CI.

A Glossary

Term	Meaning
Academic Calendar	WLU's official catalogue of courses, programs and regulations, per academic year
LORIS	WLU's registration system (Ellucian Banner); source of term offerings
CRN	Course Reference Number; identifies one section in one term
Course / CourseTerm	The stable calendar course versus one term's offering of it
RAG	Retrieval-augmented generation: answering with an LLM from retrieved evidence
Dense / sparse / multi-vector	Embedding styles: meaning (MiniLM), weighted terms (SPLADE), per-token vectors (ColBERT)
Cross-encoder	A model that scores a question and a passage together to reorder results
Retrieval profile	A named bundle of collection, search strategy, evidence budget and LLM, with optional fallback profiles for semantic search
Port / adapter	An interface defined by the domain / its implementation for a technology
Composition root	The one place where concrete adapters are chosen and wired
OIDC, PKCE	OpenID Connect sign-in; Proof Key for Code Exchange
Fail closed / fall back	On failure, deny (security state) / continue from another source (caches)
NDJSON	Newline-delimited JSON; one object per line
ETag	An HTTP validator used for conditional re-fetching

B Repository Layout

Path	Contents
`src/backend/`	FastAPI application (domain, application, infra, api, core, shared)
`src/embedding\_service/`	Embedding, sparse-vector and rerank service
`src/goldenguide_common/`	Code shared by backend and embedding service
`frontend/`	React web interface (Vite and TypeScript)
`configs/`	Layered YAML configuration (Appendix **??**)
`tests/backend/unit/`, `integration/`	Unit and integration tests mirroring the source layout
`tests/eval/`	Retrieval-quality evaluation
`docker/`	Our images: frontend, bootstrap script, development start script, BuildKit and daemon examples
`deploy/`	Configuration mounted into third-party services: Alloy, Loki, Grafana data sources, OpenTelemetry Collector, Keycloak realm
`scripts/`	Nightly ingestion client
`.github/workflows/`	Test gate and nightly ingestion
`docs/`	Documentation, including this document
`Dockerfile`, `docker-compose*.yml`	Images and the Compose stack
`.env.dev.example`, `.env.staging.example`, `.env.prod.example`	Environment variable templates; the populated files are not committed
`pyproject.toml`, `uv.lock`	Dependency groups and lockfile

C Configuration Catalogue

File	Contents
base/system_config.yml	Logging, OIDC client behaviour, job timing, service clients and scopes
base/ai_behavior.yml	Router and answer LLM prompts and defaults
base/embedding_service.yml	Embedding client: host, runtime, timeout, batch size
catalogs/providers/infra.yml	PostgreSQL, Redis, Qdrant and MinIO definitions
catalogs/providers/ml_cloud.yml	Cloud LLMs and embeddings
catalogs/providers/ml_local.yml	Local embedding, sparse and reranking models per runtime
catalogs/providers/text_splitter.yml	Chunk sizes and overlaps
catalogs/profiles/vector_collection.yml	Collection layouts and versions
catalogs/profiles/vector_retrieval.yml	Search depth, reranking and whole-page budgets
catalogs/profiles/db_evidence.yml	Structured-evidence token budgets per tier
catalogs/profiles/retrieval_profile.yml	Retrieval profiles with their fallback profiles
catalogs/active/infra.yml	Stores in use for each role
catalogs/router/router.yml	Entity lookup cache settings
catalogs/scraping/websites.yml	Websites: sitemaps, seeds, domains, selectors, exclusions
catalogs/text_enrichment/retrieval_text.yml	Enrichment templates per record type
catalogs/text_enrichment/wlu_lexicon.yml	Faculties, departments, subjects and aliases
catalogs/institution/institution.yml	Institution code, time zone, registration URL
environments/<APP_ENV>.yml	Development, staging and production overrides